```
# -*- coding: utf-8 -*-
"""Kassa-bot v4 (python-telegram-bot 21.x). Tugmalar o'sha xabarning o'zida o'zga
Guruh/kanalda bot HECH NARSA yozmaydi (faqat egasi/admin /top bersa reyting chiqa
import os
import re
import asyncio
import logging
import datetime as dt

from telegram import Update, InlineKeyboardButton, InlineKeyboardMarkup as K, Rea
from telegram.ext import (Application, CommandHandler, CallbackQueryHandler, Mess
                          ChatMemberHandler, ContextTypes, filters)

import logic as L

logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(messa
log = logging.getLogger("kassabot")

TOKEN = os.environ["BOT_TOKEN"]
OWNER = int(os.getenv("OWNER_ID", "8785896869"))
TOP_CMDS = {"/top": "td", "/topyest": "yd", "/topweek": "tw", "/toplast": "lw"}
KIND_RU = {"fine": "штраф", "bonus": "бонус", "adv": "аванс"}
DAY_RE = re.compile(r"\d{4}-\d{2}-\d{2}$")


def B(text, data):
    return InlineKeyboardButton(text, callback_data=data)


# ─────────────────────────── kirish huquqlari ───────────────────────────
def owner_ids():
    return [OWNER] + [o for o in L.owners() if o != OWNER]


def is_owner(uid):
    """Ega = OWNER_ID + egasi 'bu mening ikkinchi akkauntim' deb tasdiqlagan akka
    return uid is not None and (uid == OWNER or uid in L.owners())


def is_staff(uid):
    return is_owner(uid) or (uid is not None and uid in L.admins())
```

```
def staff_ids():
    return owner_ids() + [a for a in L.admins() if a not in owner_ids()]


def recips(wid):
    """Cashliy va Artur haqidagi hamma narsa faqat egaga."""
    w = L.worker(wid) if wid else None
    return owner_ids() if (w and w["kind"] in ("self", "partner")) else staff_ids


def can_see(uid, wid):
    w = L.worker(wid)
    return bool(w) and (is_owner(uid) or w["kind"] in ("worker", "pool"))


def vis(uid, kinds=None, active_only=True):
    return [w for w in L.workers(active_only) if can_see(uid, w["id"]) and (kinds


# ─────────────────────────── yuborish / tahrirlash ───────────────────────────
def menu_btn():
    return [B("⬅ Меню", "m:main")]


def period_rows(prefix):
    return [[B("Сегодня", f"{prefix}:td"), B("Вчера", f"{prefix}:yd")],
            [B("Эта неделя", f"{prefix}:tw"), B("Прошлая неделя", f"{prefix}:lw")
            [B("Этот месяц", f"{prefix}:tm"), B("Прошлый месяц", f"{prefix}:lm")]


def day_rows(prefix, n=6):
    t = L.today()
    ds = [t - dt.timedelta(days=i) for i in range(n)]
    return [[B(d.strftime("%d.%m"), f"{prefix}:{d.isoformat()}") for d in ds[i:i


async def send(bot, chat_id, text, kb=None):
    last = None
    parts = L.chunks(text)
    for i, part in enumerate(parts):
        try:
            last = await bot.send_message(chat_id, part, reply_markup=kb if i ==
        except Exception as e:
            log.warning("send to %s failed: %s", chat_id, e)
    return last
```

```
async def send_many(app, ids, text, kb=None):
    out = []
    for sid in ids:
        m = await send(app.bot, sid, text, kb)
        if m:
            out.append(m)
    return out


async def edit(q, text, kb=None):
    try:
        await q.edit_message_text(text, reply_markup=kb)
    except Exception as e:
        if "not modified" not in str(e).lower():
            log.warning("edit failed: %s", e)


async def show(bot, q, text, kb=None):
    """Xabarni O'SHA JOYDA o'zgartiradi. Juda uzun bo'lsa: birinchi bo'lagi o'zga
    parts = L.chunks(text)
    if len(parts) == 1:
        await edit(q, parts[0], kb)
    else:
        await edit(q, parts[0], None)
        await send(bot, q.message.chat_id, "\n".join(parts[1:]) if len(parts) > 2


async def react(bot, chat_id, msg_id):
    try:
        await bot.set_message_reaction(chat_id=chat_id, message_id=msg_id, reacti
    except Exception as e:
        log.info("reaction failed: %s", e)


MAIN_TEXT = "Главное меню.\nОтчёт вручную: просто отправьте мне суммы сюда (напри


def main_kb(uid):
    pend = len(L.pending_reports()) + len(L.pending_adjs())
    return K([[B("📊 Отчёты", "rs:menu"), B("🏆 Рейтинг", "rt:menu")],
              [B("📆 Отчёт за дату", "dt:all"), B("📄 PDF", "pdf:menu")],
              [B("👥 Обработчики", "w:list"), B("💲 Прайс", "pr:menu")],
              [B("💬 Чаты", "ch:menu"), B(f"❓ Вопросы ({pend})", "q:list")],
              [B("🛡️ Админы и доступ", "adm:menu")]])
```

```
# ———————————————————— ko'rinishlar (hammasi bir xabar ichida almashadi) ——
def is_day(key):
    return bool(DAY_RE.match(key))


def view_brief(uid, wid, key, rid=None):
    s, e = L.period_range(key)
    res = L.calc(s, e, only=wid)
    if not res or not L.has_data(res[0]) or not can_see(uid, wid):
        return None
    r, lab = res[0], L.label(s, e)
    if r["kind"] == "partner":
        text = L.zp_text(r, lab)
    else:
        text = L.brief(r, lab)
    first = [B("🔎 Разбор" if r["kind"] in ("pool", "partner") else "💰 Зп", f"zp
    rows = [first]
    if is_day(key) and r["kind"] != "partner":
        first.append(B("✏️ Чеки", f"fxd:{wid}:{key}"))
        rows.append([B("🗑️ Удалить день", f"dl:{wid}:{key}")])
        if r["kind"] == "pool":
            rows.append([B("👤 Кто работал", f"pod:{wid}:{key}")])
    if rid and r["kind"] in ("worker", "self") and L.workers(kinds=("pool",)):
        rows.append([B("🔁 Это общий аккаунт (Lazarus)", f"rl:{rid}")])
    rows.append([B("🔙 Карточка", f"w:{wid}")])
    return text, K(rows)


def view_zp(uid, wid, key, app):
    s, e = L.period_range(key)
    res = L.calc(s, e, only=wid)
    if not res or not L.has_data(res[0]) or not can_see(uid, wid):
        return None
    r, lab = res[0], L.label(s, e)
    text = L.explain(r, lab, owner=is_owner(uid))
    rows = [[B("⬅️ Кратко", f"zb:{wid}:{key}")]]
    if r["kind"] == "worker" and [u for u in L.users_of_worker(wid) if not is_sta
        rows[0].append(B("📤 Отправить", f"snd:{wid}:{key}"))
    if is_day(key) and r["kind"] != "partner":
        rows.append([B("✏️ Чеки", f"fxd:{wid}:{key}"), B("🗑️ Удалить день", f"dl:
        if r["kind"] == "pool":
            rows.append([B("👤 Кто работал", f"pod:{wid}:{key}")])
    rows.append([B("🔙 Карточка", f"w:{wid}")])
    return text, K(rows)
```

```
def card(w):
    t = L.today()
    users = L.users_of_worker(w["id"])
    tg = f"подключено аккаунтов: {len(users)}" if users else "не подключён"
    adm = " · 👑 админ" if any(u["role"] == "admin" for u in users) else ""
    st = "активен" if w["active"] else "в архиве"
    k = w["kind"]
    if k == "pool":
        return (f"🔁 {w['name']} — общий аккаунт ({st})\nЗа него работают по очер
                f"по СВОЕМУ проценту. Выбор — кнопкой «Кто работал».")
    if k == "partner":
        return f"🤝 {w['name']} — партнёр ({st})\nДоля от общей кассы: {L.money(I
    lines = [f"{'👑 ' if k == 'self' else ''}{w['name']} ({st})", f"Процент: {L.m
    for sh in L.shares_of(w["id"]):
        a, b = L.worker(sh["to_id"])["name"], L.worker(sh["from_id"])["name"]
        base = "от зп" if sh["base"] == "zp" else "от кассы"
        who = "удерживается у него" if sh["mode"] == "deduct" else "платит владел
        if sh["to_id"] == w["id"]:
            lines.append(f"🤝 Получает {L.money(sh['pct'])}% {base} с {b} ({who})
        else:
            lines.append(f"🤝 Отдаёт {L.money(sh['pct'])}% {base} → {a}")
    return "\n".join(lines)


def card_kb(w, uid):
    wid, k = w["id"], w["kind"]
    rows = [[B("📊 Сегодня", f"rep:td:{wid}"), B("📆 Вчера", f"rep:yd:{wid}")],
            [B("🗓 Эта неделя", f"rep:tw:{wid}"), B("🗓 Прошлая неделя", f"rep:lw
            [B("📈 Этот месяц", f"rep:tm:{wid}"), B("📉 Прошлый месяц", f"rep:lm:
    if k == "partner":
        rows.append([B("📆 За дату", f"dt:w:{wid}"), B("💵 Изменить долю %", f"wp
    else:
        rows.append([B("📆 За дату", f"dt:w:{wid}"), B("✏️ Исправить чеки", f"fx:
        if k == "pool":
            rows.append([B("👤 Кто работал (по дням)", f"po:{wid}")])
        else:
            rows.append([B("💵 Изменить %", f"wp:{wid}"), B("📤 Отправить работни
                         else [B("💵 Изменить %", f"wp:{wid}")])
    rows.append([B("⚙️ Ещё", f"wm:{wid}")])
    rows.append([B("BACK Обработчики", "w:list"), B("BACK Меню", "m:main")])
    return K(rows)


def more_kb(w):
    wid, k = w["id"], w["kind"]
```

```
    rows = []
    if k != "pool":
        rows += [[B("⚠️ Штраф", f"wf:{wid}"), B("🎁 Бонус", f"wb:{wid}"), B("➖ 
                 [B("💸 Выдать аванс", f"wa:{wid}"), B("↩️ Штрафы / переносы", f'
                 [B("🔀 Перенос зарплаты", f"tr1:{wid}")]]
    if k in ("worker", "self"):
        rows.append([B("🔁 Замена на день", f"sub:{wid}"), B("🤝 Доли", f"shm:{wi
    if k != "partner":
        rows.append([B("✏️ Имя", f"wn:{wid}")])
    if k == "worker":
        rows.append([B("🔄 Заменить навсегда", f"wrep:{wid}"), B("👑 Админ вкл/вы
    if k in ("worker", "pool"):
        rows.append([B("🗑️ Убрать" if w["active"] else "♻️ Вернуть", f"wrm:{wid}'
    if k != "partner":
        rows.append([B("📄 PDF эта неделя", f"pdf:w:tw:{wid}"), B("📄 PDF прошлая
    rows.append([B("🔙 Карточка", f"w:{wid}")])
    return K(rows)


async def show_card(bot, q, wid, uid):
    w = L.worker(wid)
    await edit(q, card(w), card_kb(w, uid))


async def show_acl(q, wid):
    rows = []
    for a in L.worker_adjs(wid):
        mark = "❌" if a["cancelled"] else "✅"
        rows.append([B(f"{mark} {KIND_RU.get(a['kind'], a['kind'])} {L.money(a['u
    for t in L.worker_transfers(wid):
        mark = "❌" if t["cancelled"] else "✅"
        a, b = L.worker(t["from_id"])["name"], L.worker(t["to_id"])["name"]
        rows.append([B(f"{mark} перенос {a}→{b} {L.money(t['pct'])}% · {L.dshort(
    has = bool(rows)
    rows.append([B("🔙 Ещё", f"wm:{wid}")])
    await edit(q, "Нажмите, чтобы отменить / вернуть (✅ действует, ❌ отменён):'


def price_title(r):
    rng = f"{r['lo']:,}" if r["lo"] == r["hi"] else f"{r['lo']:,}–{r['hi']:,}"
    return f"{rng} = {L.money(r['usd'])}$"


async def show_prices(q):
    rows = [[B(price_title(r), f"prc:{r['id']}")] for r in L.list_prices()]
    rows += [[B("➕ Добавить цену", "pra")], menu_btn()]
```

```
    await edit(q, "Прайс (нажмите на цену — изменить или удалить):", K(rows))


async def show_fx(q, wid, day):
    w = L.worker(wid)
    cs = L.day_checks(wid, day)
    if not cs:
        await edit(q, "На этот день чеков нет.", K([[B("➕ Чек", f"fxa:{wid}:{day
        return
    used = sum(1 for c in cs if not c["skip"])
    kb, row = [], []
    for c in cs:
        tag = ""
        if c["pay"] and c["pay"] != wid:
            pw = L.worker(c["pay"])
            tag = f"→{pw['name'][:6]}" if pw else ""
        row.append(B(("🚫 " if c["skip"] else "✅ ") + str(c["amount"]) + tag, f'
        if len(row) == 3:
            kb.append(row)
            row = []
    if row:
        kb.append(row)
    kb += [[B("📅 Дата чека", f"fxm:{wid}:{day}"), B("✏️ Сумма", f"fxe:{wid}:{day
           [B("➕ Чек", f"fxa:{wid}:{day}"), B("👤 Кто получает зп", f"fxo:{wid}:
           [B("👤 Исполнитель чека", f"fxq:{wid}:{day}"), B("📊 Разбор", f"zp:{wi
           [B("🗑️ Удалить день", f"dl:{wid}:{day}"), B("🔙 Карточка", f"w:{wid}")
    await edit(q, f"✏️ {w['name']} — {L.dshort(day)}\nЗасчитано: {used} из {len(c
                  f"Нажмите на чек: 🚫 не считать / ✅ считать.", K(kb))


async def show_admins(q):
    rows = []
    for u in L.all_users():
        wk = L.worker(u["worker_id"]) if u["worker_id"] else None
        base = (u["name"] or str(u["tg_id"])) + (f" [{wk['name']}]" if wk and wk[
        icon = "👑" if u["role"] in ("admin", "owner") else "👤"
        rows.append([B(f"{icon} {base}", f"adm:u:{u['tg_id']}")])
    rows += [[B("➕ Добавить админа по ID", "adm:new")], menu_btn()]
    await edit(q, "Администраторы и подключённые люди (нажмите на человека).\nАдмин управ
                  "кроме Cashliy и Artur (это видите только вы).", K(rows))


def safe_name(s):
    return re.sub(r"[^\w.-]", "_", s)
```

```
async def send_pdf(bot, uid, key, wid):
    import pdfgen as P
    s, e = L.period_range(key)
    lab = L.label(s, e)
    try:
        b = L.calc_bundle(s, e)
        if wid is None:
            if not any(L.has_data(r) for r in b["rows"].values()):
                await send(bot, uid, f"За {lab} данных нет.")
                return
            data = await asyncio.to_thread(P.all_pdf, b, lab, is_owner(uid))
            fname = f"obshiy_{s.isoformat()}_{e.isoformat()}.pdf"
            cap = f"📄 Общий · {lab}"
        else:
            r = b["rows"].get(wid)
            if r is None or not can_see(uid, wid) or not L.has_data(r):
                await send(bot, uid, f"За {lab} данных нет.")
                return
            data = await asyncio.to_thread(P.worker_pdf, r, lab)
            fname = f"{safe_name(r['w']['name'])}_{s.isoformat()}_{e.isoformat()}
            cap = f"📄 {r['w']['name']} · {lab}"
        await bot.send_document(uid, document=data, filename=fname, caption=cap)
    except Exception as ex:
        log.exception("pdf failed")
        await send(bot, uid, f"Не удалось создать PDF: {ex}")


# ──────────────────────── yangi hisobot: bildirishnoma va savollar ──────────
async def finalize(app, rid):
    r = L.report(rid)
    if r is None or r["status"] == "done":
        return
    L.set_report_status(rid, "done")
    wid = r["worker_id"]
    w = L.worker(wid)
    for n in L.supersede(rid):
        am = ", ".join(map(str, n["amounts"]))
        if n["kind"] == "replace":
            await send_many(app, recips(wid), f"♻️ {w['name']}: прежний отчёт ({a
                            K([[B("↩️ Вернуть старый", f"rrs:{n['old']}")]]))
        else:
            await send_many(app, recips(wid), f"⚠️ {w['name']}: чек {am} уже был.
                            K([[B("Да, убрать новый", f"rdup:{rid}"), B("Нет, оба
    for d in L.report_days(rid):
        for sid in recips(wid):
            v = view_brief(sid, wid, d, rid)
```

```
            if v:
                await send(app.bot, sid, v[0], v[1])


async def advance(app, rid):
    r = L.report(rid)
    if r is None:
        return
    need = L.report_needs(rid)
    if need is None:
        await finalize(app, rid)
        return
    kind, arg = need
    snippet = (r["raw"] or "")[:350]
    if kind == "worker":
        who = f"\nОтправитель: {r['sender_name']}" if r["sender_name"] else ""
        for sid in staff_ids():
            rows = [[B(w["name"], f"pw:{rid}:{w['id']}")] for w in vis(sid, ("wor
            rows.append([B("🗄 Из архива", f"pwa:{rid}"), B("🚫 Это не отчёт", f'
            await send(app.bot, sid, f"❓ Чей это отчёт?{who}\n\n{snippet}", K(rc
    elif kind == "date":
        rows = day_rows(f"pd:{rid}", 4)
        rows.append([B("📅 Другая дата", f"pdo:{rid}"), B("🚫 Это не отчёт", f"pv
        name = L.worker(r["worker_id"])["name"]
        await send_many(app, recips(r["worker_id"]), f"❓ {name}: на какой день з
    elif kind == "price":
        name = L.worker(r["worker_id"])["name"]
        msgs = await send_many(app, recips(r["worker_id"]),
                               f"❓ {name}: сколько $ стоит чек {arg:,}? В прайсе
                               f"Ответьте на это сообщение числом (например 12).
        for m in msgs:
            app.bot_data.setdefault("pq", {})[(m.chat_id, m.message_id)] = (rid,
    elif kind == "op":
        name = L.worker(r["worker_id"])["name"]
        days = ", ".join(L.dshort(d) for d in L.report_days(rid))
        for sid in staff_ids():
            rows = [[B(w["name"], f"op:{rid}:{w['id']}")] for w in vis(sid, ("wor
            rows.append([B("🚫 Это не отчёт", f"pwi:{rid}")])
            await send(app.bot, sid, f"❓ Кто работал на {name} ({days})?\nЗп пол


async def ask_unassigned(app):
    """Umumiy akkauntning hali kim ishlagani belgilanmagan kunlari."""
    for pw in L.workers(active_only=False, kinds=("pool",)):
        for d in L.unassigned_days(pw["id"]):
            for sid in staff_ids():
```

```
                rows = [[B(w["name"], f"pod2:{pw['id']}:{d}:{w['id']}")] for w in
                await send(app.bot, sid, f"❓ Кто работал на {pw['name']} {L.dshc


# ──────────────────────── shtraf / bonus / avans: DOIM so'raydi ────────────────
def adj_title(a):
    w = L.worker(a["worker_id"])["name"] if a["worker_id"] else "?"
    k = KIND_RU.get(a["kind"], "штраф/бонус/аванс")
    return f"{w} — {k} {L.money(a['usd'])}$ ({L.dshort(a['day'])})"


def adj_view(aid, uid):
    """-> (text, kb) — savolning hozirgi bosqichi."""
    a = L.adj(aid)
    snippet = (a["raw"] or "")[:300]
    if a["worker_id"] is None:
        rows = [[B(w["name"], f"aw:{aid}:{w['id']}")] for w in vis(uid, ("worker"
        rows.append([B("❌ Отмена", f"ai:{aid}")])
        return f"❓ Кому {KIND_RU.get(a['kind'], 'штраф/бонус/аванс')} {L.money(a
    if a["kind"] is None:
        rows = [[B("Штраф", f"ak:{aid}:fine"), B("Бонус", f"ak:{aid}:bonus"), B("
        return f"❓ {L.worker(a['worker_id'])['name']}: {L.money(a['usd'])}$ — шт
    return (f"❓ Подтвердите: {adj_title(a)}\n\nСообщение: {snippet}",
            K([[B("✅ Верно", f"ay:{aid}"), B("❌ Отмена", f"ai:{aid}")], [B("✏️


async def ask_adj(app, aid):
    a = L.adj(aid)
    if a is None:
        return
    for sid in (recips(a["worker_id"]) if a["worker_id"] else staff_ids()):
        text, kb = adj_view(aid, sid)
        await send(app.bot, sid, text, kb)


async def handle_adjust(app, text, day, chat_id, msg_id):
    items, _clean = L.parse_adjustments(text)
    if not items:
        await send_many(app, staff_ids(), f"❓ Штраф/бонус/аванс увидел, но сумму
        return
    for it in items:                                    # HAMMASI tasdiqlanmagunc
        aid = L.add_adj(it["worker_id"], it["kind"], it["usd"], L.iso(day), text,
        await ask_adj(app, aid)


# ──────────────────────── guruh / kanal ────────────────────────
```

```
async def group_top(app, chat_id, text):
    cmd = text.split()[0].split("@")[0].lower()
    if cmd in TOP_CMDS:
        s, e = L.period_range(TOP_CMDS[cmd])
        await send(app.bot, chat_id, L.rating_text(L.calc_bundle(s, e), L.label(s


async def submit_report(app, acct, op, chat_id, msg_id, sender_id, sname, text, c
    rid = L.create_report(acct, chat_id, msg_id, sender_id, text, checks, sname)
    if acct and op and op != acct:                      # boshqa akkauntda ishlag
        L.set_pay_report(rid, op)
    await advance(app, rid)
    return rid


async def on_chat_text(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    msg, chat = update.effective_message, update.effective_chat
    if not msg or not chat:
        return
    text = (msg.text or msg.caption or "").strip()
    if not text:
        return
    app = ctx.application
    u = msg.from_user
    uid = u.id if u else None
    anon = uid is None or uid == L.ANON or msg.sender_chat is not None
    staff = anon or is_staff(uid)
    edited = bool(update.edited_message or update.edited_channel_post)

    row = L.chat(chat.id)
    if row is None:
        L.upsert_chat(chat.id, chat.title or str(chat.id))
        if staff:
            L.set_watch(chat.id, 1)
            row = L.chat(chat.id)
        else:
            await send(app.bot, OWNER, f"➕ Новый чат «{chat.title}». Читать там
                       K([[B("✅ Следить", f"cw:{chat.id}:1"), B("🚫 Нет", f"cw:{
            return
    if not row["watch"]:
        return
    if text.startswith("/"):
        if staff and not edited:
            await group_top(app, chat.id, text)
        return
```

```
    day = L.post_day(msg.date)

    # shtraf / bonus / avans — faqat egasi/admin yozganda, har doim lichkada tasd
    if staff and L.has_kind(text):
        if edited:
            L.drop_adj_msg(chat.id, msg.message_id)
        await handle_adjust(app, text, day, chat.id, msg.message_id)
        return

    # hisobot: faqat raqamlardan iborat (ixtiyoriy birinchi qator — akkaunt nomi)
    tag, body = L.split_tag(text)
    if not L.is_report_like(body):
        return
    poster = None
    if is_owner(uid):
        poster = L.self_worker()["id"]
    elif uid and not anon:
        ur = L.user(uid)
        if ur and ur["worker_id"]:
            poster = ur["worker_id"]
            L.touch_user(uid, u.full_name)
    checks = L.report_checks(body, day, is_owner(uid))
    if not checks:
        return
    await react(ctx.bot, chat.id, msg.message_id)
    acct = tag or poster
    sname = (u.full_name + (f" @{u.username}" if u.username else "")) if u else "
    old = L.report_amounts_by_msg(chat.id, msg.message_id) if edited else None
    if old is not None:
        nm = L.worker(acct)["name"] if acct else (sname or "?")
        await send_many(app, recips(acct), f"✏️ {nm} изменил отчёт:\nбыло: {', '.
                                            f"стало: {', '.join(str(a) for _, a in
    await submit_report(app, acct, poster if tag else None, chat.id, msg.message_


async def on_my_chat_member(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    cm = update.my_chat_member
    chat = cm.chat
    if chat.type == "private":
        return
    st = cm.new_chat_member.status
    if st in ("member", "administrator"):
        L.upsert_chat(chat.id, chat.title or str(chat.id))
        if is_staff(cm.from_user.id if cm.from_user else None):
            L.set_watch(chat.id, 1)
            await send(ctx.bot, OWNER, f"✅ Слежу за отчётами в «{chat.title}».")
```

```
        elif not L.chat(chat.id)["watch"]:
            await send(ctx.bot, OWNER, f"➕ Бота добавили в «{chat.title}» (добав
                       K([[B("✅ Следить", f"cw:{chat.id}:1"), B("🚪 Выйти из чат
    elif st in ("left", "kicked"):
        L.upsert_chat(chat.id, chat.title or str(chat.id))
        L.set_watch(chat.id, 0)


# ──────────────────────── shaxsiy chat: /start ────────────────────────
async def cmd_start(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    u = update.effective_user
    if update.effective_chat.type != "private":
        return
    ctx.user_data.pop("st", None)
    if is_staff(u.id):
        await update.message.reply_text(MAIN_TEXT, reply_markup=main_kb(u.id))
        return
    if L.user(u.id):
        return                                          # qabul qilingan oddiy fo
    ctx.application.bot_data.setdefault("names", {})[u.id] = u.full_name
    asked = ctx.application.bot_data.setdefault("asked", set())
    if u.id in asked:
        return
    asked.add(u.id)                                     # begonaga HECH NARSA ko'
    uname = f"@{u.username}" if u.username else "без username"
    for sid in staff_ids():
        rows = [[B(w["name"], f"ul:{u.id}:{w['id']}")] for w in vis(sid, ("worker
        rows.append([B("➕ Новое имя", f"un:{u.id}")])
        if sid == OWNER:
            rows.append([B("👑 Это мой Cashliy-аккаунт (я)", f"uo:{u.id}")])
        rows.append([B("👑 Админ", f"ua:{u.id}"), B("🚫 Отказать", f"ux:{u.id}")])
        await send(ctx.bot, sid, f"🆕 Запрос доступа: {u.full_name} ({uname}, ID
                                 f"Привязать к обработчику (ему будет уходить тол


async def cmd_id(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    if update.effective_chat.type == "private":
        await update.message.reply_text(f"Ваш ID: {update.effective_user.id}")


# ──────────────────────── tugmalar (hammasi o'sha xabarning o'zida) ──────────
def payee_rows(uid, prefix, back, include_owner_btn=False, wid=None):
    rows = [[B(w["name"], f"{prefix}:{w['id']}")] for w in vis(uid, ("worker", "s
    if include_owner_btn:
        rows.append([B("↩️ Хозяин аккаунта (как было)", f"{prefix}:0")])
    rows.append([B("🔙 Назад", back)])
```

```
    return rows


async def on_cb(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    q = update.callback_query
    uid = q.from_user.id
    d = q.data
    p = d.split(":")
    await q.answer()
    if not is_staff(uid):
        return                                          # begona va oddiy foydala
    app, bot = ctx.application, ctx.bot
    A = p[0]

    if d == "m:main":
        ctx.user_data.pop("st", None)
        await edit(q, MAIN_TEXT, main_kb(uid))
    elif d == "okk":
        await edit(q, "Хорошо, оставил оба.")

    # ── hisobotlar
    elif d == "rs:menu":
        await edit(q, "За какой период отчёт?", K(period_rows("rs")))
    elif A == "rs":
        s, e = L.period_range(p[1])
        lab = L.label(s, e)
        b = L.calc_bundle(s, e)
        rows_ = [b["rows"][i] for i in b["order"] if L.has_data(b["rows"][i]) and
        if not any(L.has_data(r) and can_see(uid, r["w"]["id"]) for r in b["rows"
            await edit(q, f"За {lab} отчётов нет.", K([[B("⬅ Периоды", "rs:menu"
            return
        kb = [[B(r["w"]["name"], f"rep:{p[1]}:{r['w']['id']}")] for r in rows_]
        kb += [[B("⬅ Периоды", "rs:menu"), B("📄 PDF", "pdf:menu")]]
        await show(bot, q, L.overall_text(b, lab, is_owner(uid)), K(kb))
    elif A == "rep":
        key, wid = p[1], int(p[2])
        v = view_brief(uid, wid, key)
        if not v:
            s, e = L.period_range(key)
            await edit(q, f"{L.worker(wid)['name']}: за {L.label(s, e)} отчётов н
        else:
            await show(bot, q, *v)
    elif A == "zp":
        wid, key = int(p[1]), p[2]
        v = view_zp(uid, wid, key, app)
        if v:
```

```
            await show(bot, q, *v)
        else:
            await edit(q, "Данных нет.", K([[B("⬅️ Карточка", f"w:{wid}")]]))
    elif A == "zb":
        v = view_brief(uid, int(p[1]), p[2])
        if v:
            await show(bot, q, *v)
    elif d == "rt:menu":
        await edit(q, "Рейтинг за какой период?", K(period_rows("rat")))
    elif A == "rat":
        s, e = L.period_range(p[1])
        await show(bot, q, L.rating_text(L.calc_bundle(s, e), L.label(s, e)), K([
    elif d == "dt:all":
        ctx.user_data["st"] = ("datepick", None)
        await edit(q, "Напишите дату (05.10) или период (05.10-11.10):", K([menu_
    elif A == "dt" and p[1] == "w":
        ctx.user_data["st"] = ("datepick", int(p[2]))
        await edit(q, "Напишите дату (05.10) или период (05.10-11.10):", K([menu_
    elif A == "dl":
        wid, day = int(p[1]), p[2]
        await edit(q, f"Удалить ВСЕ чеки {L.worker(wid)['name']} за {L.dshort(day
                   K([[B("✅ Да, удалить", f"dy:{wid}:{day}"), B("Нет", f"zb:{wid
    elif A == "dy":
        L.clear_day(int(p[1]), p[2])
        await edit(q, "🗑 День удалён.", K([[B("⬅️ Карточка", f"w:{p[1]}")]]))

    # ── ishchiga yuborish: avval ko'rasiz, keyin tasdiqlaysiz
    elif A == "pv":
        wid = p[1]
        await edit(q, "Какой период отправить обработчику?", K([[B("Сегодня", f"s
                                                                 [B("Эта неделя", f"
                                                                 [B("⬅️ Назад", f"w:
    elif A in ("snd", "sdy"):
        wid, key = int(p[1]), p[2]
        s, e = L.period_range(key)
        res = L.calc(s, e, only=wid)
        users = [u for u in L.users_of_worker(wid) if not is_staff(u["tg_id"])]
        w = L.worker(wid)
        if not res or not L.has_data(res[0]) or w["kind"] != "worker":
            await edit(q, "Нечего отправлять: данных нет.", K([[B("⬅️ Карточка",
            return
        if not users:
            await edit(q, f"{w['name']} не подключён к Telegram (пусть нажмёт /st
            return
        text = L.zp_text(res[0], L.label(s, e))
        prev = app.bot_data.setdefault("prev", {})
```

```
    sent = app.bot_data.setdefault("sent", set())
    if A == "snd" or prev.get((wid, key)) != text:
        prev[(wid, key)] = text
        note = "" if A == "snd" else "⚠️ Данные изменились. "
        await edit(q, f"{note}👁️ Вот что получит {w['name']} (пока НЕ отправл
                   K([[B("✅ Отправить", f"sdy:{wid}:{key}"), B("⬅️ Назад", f
    else:
        if (wid, key, text) in sent:
            await edit(q, f"✅ Уже отправлено {w['name']}:\n\n{text}")
            return
        for u in users:
            await send(bot, u["tg_id"], text)
        sent.add((wid, key, text))
        await edit(q, f"✅ Отправлено {w['name']}:\n\n{text}", K([[B("⬅️ Карт

# ── obrabotchiklar
elif d == "w:list":
    rows = []
    for w in vis(uid):
        icon = {"self": "👑 ", "partner": "🤝 ", "pool": "🔁 "}.get(w["kind"]
        pc = "" if w["kind"] == "pool" else f" · {L.money(L.percent_on(w['id'
        rows.append([B(f"{icon}{w['name']}{pc}", f"w:{w['id']}")])
    rows += [[B("➕ Новый", "w:new"), B("🗄️ Архив", "w:arch")], menu_btn()]
    await edit(q, "Обработчики:", K(rows))
elif d == "w:arch":
    arch = [w for w in vis(uid, active_only=False) if not w["active"]]
    await edit(q, "Архив:" if arch else "Архив пуст.", K([[B(w["name"], f"w:{
elif d == "w:new":
    ctx.user_data["st"] = ("newname",)
    await edit(q, "Напишите имя нового обработчика:", K([menu_btn()]))
elif A == "w" and p[1].isdigit():
    if can_see(uid, int(p[1])):
        await show_card(bot, q, int(p[1]), uid)
elif A == "wm":
    w = L.worker(int(p[1]))
    await edit(q, f"⚙️ {w['name']} — ещё:", more_kb(w))
elif A == "wp":
    ctx.user_data["st"] = ("pct", int(p[1]))
    await edit(q, "Напишите новый процент числом (например 41):", K([menu_btn
elif A == "pcd":
    wid, pct, when = int(p[1]), float(p[2]), p[3]
    t = L.today()
    frm = {"mw": L.week_start(t), "td": t, "tm": t + dt.timedelta(days=1)}.ge
    L.set_percent(wid, pct, L.iso(frm))
    await show_card(bot, q, wid, uid)
elif A in ("wf", "wb", "wbm", "wa"):
```

```
    wid = int(p[1])
    ctx.user_data["st"] = ({"wf": "fine", "wb": "bonus", "wbm": "bonusminus",
    what = {"wf": "штрафа", "wb": "бонуса", "wbm": "бонуса, который нужно выч
    await edit(q, f"Напишите сумму {what} в $ (например 5):", K([menu_btn()])
elif A == "acl":
    await show_acl(q, int(p[1]))
elif A == "act":
    L.adj_toggle(int(p[1]))
    await show_acl(q, int(p[2]))
elif A == "trt":
    L.transfer_toggle(int(p[1]))
    await show_acl(q, int(p[2]))
elif A == "wn":
    ctx.user_data["st"] = ("rename", int(p[1]))
    await edit(q, "Напишите новое имя:", K([menu_btn()]))
elif A == "wrep":
    w = L.worker(int(p[1]))
    ctx.user_data["st"] = ("replace", w["id"])
    await edit(q, f"Кто будет работать вместо {w['name']} постоянно? Напишите
                  f"({L.money(L.percent_on(w['id'], L.today()))}%), его можно
               K([menu_btn()]))
elif A == "wad":
    wid = int(p[1])
    us = L.users_of_worker(wid)
    if not us:
        await edit(q, "Сначала обработчик должен нажать /start у бота, а вы —
    else:
        for u in us:
            new = "worker" if u["role"] == "admin" else "admin"
            L.set_role(u["tg_id"], new)
            await send(bot, u["tg_id"], "👑 Вам выданы права администратора. Нажмите
        await show_card(bot, q, wid, uid)
elif A == "wrm":
    w = L.worker(int(p[1]))
    await edit(q, f"Убрать {w['name']} в архив? История сохранится.", K([[B("
elif A == "wrmy":
    L.set_active(int(p[1]), 0)
    await edit(q, "Убран в архив.", K([[B("⬅️ Обработчики", "w:list")]]))
elif A == "wre":
    L.set_active(int(p[1]), 1)
    await show_card(bot, q, int(p[1]), uid)

# ── oylik o'tkazish (Cashliy va Artur ham)
elif A == "tr1":
    await edit(q, f"🔀 Перенос зарплаты {L.worker(int(p[1]))['name']}. За как
               K(day_rows(f"tr2:{p[1]}", 6) + [[B("⬅️ Ещё", f"wm:{p[1]}")]]))
```

```
elif A == "tr2":
    wid, day = int(p[1]), p[2]
    base = L.day_base(wid, day)
    if base <= 0:
        await edit(q, f"У {L.worker(wid)['name']} нет зарплаты за {L.dshort(d
    else:
        rows = [[B(w["name"], f"tr3:{wid}:{day}:{w['id']}")] for w in vis(uid
        rows.append([B("⬅️ Назад", f"tr1:{wid}")])
        await edit(q, f"Зп за {L.dshort(day)}: {L.money(base)}$. Кому перенес
elif A == "tr3":
    wid, day, to = int(p[1]), p[2], int(p[3])
    rows = [[B("100%", f"tr4:{wid}:{day}:{to}:100"), B("75%", f"tr4:{wid}:{da
             [B("50%", f"tr4:{wid}:{day}:{to}:50"), B("25%", f"tr4:{wid}:{day}
             [B("✏️ Другой %", f"tro:{wid}:{day}:{to}")], [B("⬅️ Назад", f"tr1
    await edit(q, f"Какую часть зп за {L.dshort(day)} перенести → {L.worker(t
elif A == "tro":
    ctx.user_data["st"] = ("trpct", int(p[1]), p[2], int(p[3]))
    await edit(q, "Напишите процент переноса числом (например 40):", K([menu_
elif A == "tr4":
    wid, day, to, pct = int(p[1]), p[2], int(p[3]), float(p[4])
    tid = L.add_transfer(day, wid, to, pct)
    await edit(q, f"✅ Перенос: {L.worker(wid)['name']} → {L.worker(to)['name
                  f"{L.money(L.r2(L.day_base(wid, day) * pct / 100))}$", K([[

# ── замена на день: kim ishlagan bo'lsa oylikni o'sha oladi
elif A == "sub":
    days = L.worker_days(int(p[1]), 10)
    rows = [[B(f"{L.dshort(r['rd'])} ({r['n']} {L.chk_word(r['n'])})", f"sub2
    await edit(q, "🔁 Замена на день: какой день?" if days else "Чеков пока н
elif A == "sub2":
    wid, day = int(p[1]), p[2]
    rows = [[B(w["name"], f"sub3:{wid}:{day}:{w['id']}")] for w in vis(uid, (
    await edit(q, f"Кто работал вместо {L.worker(wid)['name']} {L.dshort(day)
               K(rows + [[B("⬅️ Назад", f"sub:{wid}")]]))
elif A == "sub3":
    wid, day, pid = int(p[1]), p[2], int(p[3])
    L.set_pay_day(wid, day, pid)
    await edit(q, f"✅ {L.dshort(day)}: кассу {L.worker(wid)['name']} отрабат
               K([[B("↩️ Вернуть как было", f"subr:{wid}:{day}"), B("💰 Зп",
elif A == "subr":
    L.set_pay_day(int(p[1]), p[2], None)
    await edit(q, "↩️ Вернул: зп получает владелец аккаунта.", K([[B("⬅️ Карт

# ── umumiy akkaunt (Lazarus): kim ishladi
elif A == "po":
    pid = int(p[1])
```

```
    days = L.worker_days(pid, 10)
    rows = [[B(f"{L.dshort(r['rd'])} ({r['n']} {L.chk_word(r['n'])})", f"pod:
    await edit(q, f"👤 {L.worker(pid)['name']}: кто работал — какой день?" if
elif A == "pod":
    pid, day = int(p[1]), p[2]
    pool = L.worker(pid)
    rows = [[B(w["name"], f"pod2:{pid}:{day}:{w['id']}")] for w in vis(uid, (
    if pool["kind"] != "pool":
        rows.append([B("↩️ Хозяин аккаунта", f"pod2:{pid}:{day}:0")])
    await edit(q, f"Кто работал на {pool['name']} {L.dshort(day)}?\nЗп получи
elif A == "pod2":
    pid, day, who = int(p[1]), p[2], int(p[3])
    L.set_pay_day(pid, day, who or None)
    await edit(q, f"✅ {L.dshort(day)}: {L.worker(pid)['name']} — работал {L.
                K([[B("📊 Разбор", f"zp:{pid}:{day}")], [B("🔙 Карточка", f"w:
elif A == "op":
    rid, who = int(p[1]), int(p[2])
    if not L.report(rid) or L.report_needs(rid) != ("op", None):
        await edit(q, "Уже решено.")
        return
    L.set_pay_report(rid, who)
    await edit(q, f"✅ Работал: {L.worker(who)['name']}")
    await advance(app, rid)
elif A == "rl":                                    # hisobot aslida umumiy a
    r = L.report(int(p[1]))
    pools = L.workers(kinds=("pool",))
    if r and pools:
        old = r["worker_id"]
        L.x("update reports set worker_id=? where id=?", (pools[0]["id"], r["
        L.x("update checks set pay=? where report_id=?", (old, r["id"]))
        await edit(q, f"🔁 Этот отчёт теперь — {pools[0]['name']}, работал: {
                  K([[B("📊 Разбор", f"zp:{pools[0]['id']}:{L.report_days(r[

# ── ulushlar (Bayzed <- Ваха 5%)
elif A == "shm":
    wid = int(p[1])
    rows = []
    for sh in L.shares_of(wid):
        a, b_ = L.worker(sh["to_id"])["name"], L.worker(sh["from_id"])["name"
        rows.append([B(f"{a} ← {b_} {L.money(sh['pct'])}% {'от зп' if sh['bas
    rows += [[B(f"➕ Новая доля: {L.worker(wid)['name']} получает с кого-то",
    await edit(q, "🤝 Доли (кто с кого получает процент):", K(rows))
elif A == "shc":
    sh = L.share_row(int(p[1]))
    a, b_ = L.worker(sh["to_id"])["name"], L.worker(sh["from_id"])["name"]
    await edit(q, f"🤝 {a} получает {L.money(sh['pct'])}% {'от зарплаты' if s
```

```
                    f"Кто платит: {'удерживается у ' + b_ if sh['mode'] == 'ded
                    f"Действует с: {L.dshort(sh['from_day'])}",
                K([[B("💯 Изменить %", f"shp:{sh['id']}")],
                    [B("🔄 База: зп ↔ касса", f"shb:{sh['id']}")],
                    [B("💳 Платит: удерживать ↔ владелец", f"shd:{sh['id']}")],
                    [B("🗑 Удалить", f"shx:{sh['id']}")], [B("🔙 Назад", f"shm:
elif A == "shp":
    ctx.user_data["st"] = ("shpct", int(p[1]))
    await edit(q, "Напишите новый процент доли числом (например 5):", K([menu
elif A in ("shb", "shd"):
    sh = L.share_row(int(p[1]))
    if A == "shb":
        L.update_share(sh["id"], base="kassa" if sh["base"] == "zp" else "zp"
    else:
        L.update_share(sh["id"], mode="owner" if sh["mode"] == "deduct" else
    sh = L.share_row(sh["id"])
    a, b_ = L.worker(sh["to_id"])["name"], L.worker(sh["from_id"])["name"]
    await edit(q, f"✅ Изменено.\n🤝 {a} получает {L.money(sh['pct'])}% {'от
                    f"Кто платит: {'удерживается у ' + b_ if sh['mode'] == 'ded
                K([[B("🔙 К доле", f"shc:{sh['id']}")], [B("🔙 Назад", f"shm:{
elif A == "shx":
    await edit(q, "Удалить эту долю?", K([[B("✅ Да", f"shxy:{p[1]}"), B("Нет
elif A == "shxy":
    sh = L.share_row(int(p[1]))
    L.del_share(sh["id"])
    await edit(q, "🗑 Доля удалена.", K([[B("🔙 Назад", f"shm:{sh['to_id']}")
elif A == "shn":
    wid = int(p[1])
    rows = [[B(w["name"], f"shn2:{wid}:{w['id']}")] for w in vis(uid, ("worke
    await edit(q, f"С кого {L.worker(wid)['name']} получает процент?", K(rows
elif A == "shn2":
    ctx.user_data["st"] = ("shnew", int(p[1]), int(p[2]))
    await edit(q, "Сколько процентов? Напишите число (например 5). По умолчан

# ── chek tuzatish (hamma uchun, jumladan Cashliy)
elif A == "fx":
    wid = int(p[1])
    days = L.worker_days(wid, 14)
    rows = [[B(f"{L.dshort(r['rd'])} ({r['n']} {L.chk_word(r['n'])})", f"fxd:
    await edit(q, "Выберите день:" if days else "Чеков пока нет.", K(rows + [
elif A == "fxd":
    await show_fx(q, int(p[1]), p[2])
elif A == "fxc":
    cid, wid, day = int(p[1]), int(p[2]), p[3]
    c = L.check_row(cid)
    if c and c["acct"] == wid:
```

```
        L.toggle_check(cid)
    await show_fx(q, wid, day)
elif A == "fxm":
    wid, day = int(p[1]), p[2]
    rows = [[B(f"{c['amount']}", f"fxm2:{c['id']}:{wid}:{day}")] for c in L.d
    await edit(q, "У какого чека изменить дату?", K(rows + [[B("⬅ Назад", f"
elif A == "fxm2":
    cid, wid, day = int(p[1]), int(p[2]), p[3]
    rows = day_rows(f"fxm3:{cid}:{wid}:{day}", 6)
    await edit(q, "Новая дата чека:", K(rows + [[B("⬅ Назад", f"fxd:{wid}:{d
elif A == "fxm3":
    cid, wid, day, new = int(p[1]), int(p[2]), p[3], p[4]
    c = L.check_row(cid)
    if c and c["acct"] == wid:
        L.set_check_day(cid, new)
    await show_fx(q, wid, day)
elif A == "fxe":
    wid, day = int(p[1]), p[2]
    rows = [[B(f"{c['amount']}", f"fxe2:{c['id']}:{wid}:{day}")] for c in L.d
    await edit(q, "У какого чека изменить сумму?", K(rows + [[B("⬅ Назад", f
elif A == "fxe2":
    ctx.user_data["st"] = ("chkamt", int(p[1]), int(p[2]), p[3])
    await edit(q, "Напишите новую сумму чека (например 1950):", K([menu_btn()
elif A == "fxa":
    ctx.user_data["st"] = ("fxadd", int(p[1]), p[2])
    await edit(q, "Напишите сумму нового чека (например 1950):", K([menu_btn(
elif A == "fxo":
    wid, day = int(p[1]), p[2]
    await edit(q, f"Кто получает зп за {L.dshort(day)} (весь день этого аккау
               K(payee_rows(uid, f"fxo2:{wid}:{day}", f"fxd:{wid}:{day}", inc
elif A == "fxo2":
    wid, day, who = int(p[1]), p[2], int(p[3])
    L.set_pay_day(wid, day, who or None)
    await show_fx(q, wid, day)
elif A == "fxq":
    wid, day = int(p[1]), p[2]
    rows = [[B(f"{c['amount']}", f"fxq2:{c['id']}:{wid}:{day}")] for c in L.d
    await edit(q, "Какому чеку выбрать исполнителя (кто получит зп)?", K(rows
elif A == "fxq2":
    cid, wid, day = int(p[1]), int(p[2]), p[3]
    await edit(q, "Кто получает зп с этого чека?", K(payee_rows(uid, f"fxq3:{
elif A == "fxq3":
    cid, wid, day, who = int(p[1]), int(p[2]), p[3], int(p[4])
    c = L.check_row(cid)
    if c and c["acct"] == wid:
        L.set_pay_check(cid, who or None)
```

```
        await show_fx(q, wid, day)
    elif A == "rrs":
        L.set_report_status(int(p[1]), "done")
        await edit(q, "↩️ Старый отчёт возвращён (теперь считаются оба).")
    elif A == "rdup":
        L.set_report_status(int(p[1]), "replaced")
        await edit(q, "✅ Повтор убран.")

    # ── prays
    elif d == "pr:menu":
        await show_prices(q)
    elif A == "prc":
        r = L.price_row(int(p[1]))
        if r:
            await edit(q, f"Цена: {price_title(r)}", K([[B("✏️ Изменить", f"pre:{
    elif A == "pre":
        ctx.user_data["st"] = ("pricedit", int(p[1]))
        await edit(q, "Напишите новую цену: «сумма цена» (5200 20) или «от до цен
    elif A == "prd":
        r = L.price_row(int(p[1]))
        await edit(q, f"Удалить цену {price_title(r)}?", K([[B("✅ Да, удалить",
    elif A == "prdy":
        L.del_price(int(p[1]))
        await show_prices(q)
    elif d == "pra":
        ctx.user_data["st"] = ("priceadd",)
        await edit(q, "Напишите: «сумма цена» (5200 20) или «от до цена» (5000 54

    # ── PDF
    elif d == "pdf:menu":
        await edit(q, "PDF за какой период?", K([[B("Эта неделя", "pdf:p:tw"), B(
                                                 [B("Этот месяц", "pdf:p:tm"), B
    elif A == "pdf" and p[1] == "p":
        rows = [[B("📚 Общий + каждый отдельно", f"pdf:e:{p[2]}")], [B("📦 Только
        rows += [[B(f"📄 {w['name']}", f"pdf:w:{p[2]}:{w['id']}")] for w in vis(u
        await edit(q, "Что отправить?", K(rows + [[B("⬅️ Назад", "pdf:menu")]]))
    elif A == "pdf" and p[1] == "a":
        await send_pdf(bot, uid, p[2], None)
    elif A == "pdf" and p[1] == "e":
        s, e = L.period_range(p[2])
        await send_pdf(bot, uid, p[2], None)
        b = L.calc_bundle(s, e)
        for i in b["order"]:
            if L.has_data(b["rows"][i]) and can_see(uid, i):
                await send_pdf(bot, uid, p[2], i)
    elif A == "pdf" and p[1] == "w":
```

```
    await send_pdf(bot, uid, p[2], int(p[3]))

# ── chatlar
elif d == "ch:menu":
    rows = [[B(("✅ " if c["watch"] else "⏸️ ") + (c["title"] or str(c["chat_
    await edit(q, "Группы и каналы, где бот читает отчёты. Нажмите — включить
                  "Новый чат: просто добавьте бота в группу/канал — включится
elif A == "cw":
    L.set_watch(int(p[1]), int(p[2]))
    await edit(q, "✅ Слежу за отчётами." if p[2] == "1" else "Хорошо, не сле
elif A == "cl":
    try:
        await bot.leave_chat(int(p[1]))
    except Exception:
        pass
    L.set_watch(int(p[1]), 0)
    await edit(q, "🚪 Вышел из чата.")

# ── savollar (birinchi javob beruvchi yutadi)
elif d == "q:list":
    pr, pa = L.pending_reports(), L.pending_adjs()
    if not pr and not pa:
        await edit(q, "Открытых вопросов нет 😊", K([menu_btn()]))
    else:
        await edit(q, f"Повторяю {len(pr) + len(pa)} вопрос(а) ниже 👇", K([m
        for rid in pr:
            await advance(app, rid)
        for aid in pa:
            await ask_adj(app, aid)
        await ask_unassigned(app)
elif A == "pwa":
    arch = [w for w in vis(uid, ("worker", "pool"), active_only=False) if not
    rows = [[B(w["name"], f"pw:{p[1]}:{w['id']}")] for w in arch] or [[B("(ap
    await edit(q, "Из архива:", K(rows))
elif A == "pw":
    rid, wid = int(p[1]), int(p[2])
    r = L.report(rid)
    if not r or r["worker_id"] is not None:
        await edit(q, "Уже решено.")
        return
    L.x("update reports set worker_id=? where id=?", (wid, rid))
    sid = r["sender_id"]
    if sid and not is_staff(sid) and not L.user(sid) and sid != L.ANON and L.
        L.upsert_user(sid, r["sender_name"] or "", "worker", wid)       # key
    await edit(q, f"✅ Отчёт: {L.worker(wid)['name']}")
    await advance(app, rid)
```

```
elif A == "pd":
    rid = int(p[1])
    if not L.report(rid) or not L.q1("select 1 from checks where report_id=?
        await edit(q, "Уже решено.")
        return
    L.x("update checks set rd=? where report_id=? and rd is null", (p[2], rid
    await edit(q, f"✅ Дата: {L.dshort(p[2])}")
    await advance(app, rid)
elif A == "pdo":
    ctx.user_data["st"] = ("pdate", int(p[1]))
    await edit(q, "Напишите дату, например 05.10 или 05.10.2026:")
elif A == "pwi":
    L.delete_report(int(p[1]))
    await edit(q, "🚫 Игнорирую это сообщение.")

# ── shtraf / bonus / avans tasdiqlash
elif A in ("aw", "ak", "ay", "ai", "ae"):
    a = L.adj(int(p[1]))
    if A == "ai":
        if a:
            L.adj_delete(a["id"])
        await edit(q, "❌ Отменено — не засчитано.")
        return
    if not a or not a["pending"]:
        await edit(q, "Уже решено.")
        return
    if A == "aw":
        L.adj_set(a["id"], worker_id=int(p[2]))
    elif A == "ak":
        L.adj_set(a["id"], kind=p[2])
    elif A == "ae":
        ctx.user_data["st"] = ("adjamt", a["id"])
        await edit(q, "Напишите правильную сумму в $:", K([menu_btn()]))
        return
    elif A == "ay":
        L.adj_set(a["id"], pending=0)
        await edit(q, f"✅ Записано: {adj_title(L.adj(a['id']))}", K([[B("↩️
        return
    text, kb = adj_view(a["id"], uid)
    await edit(q, text, kb)
elif A == "ac":
    a = L.adj(int(p[1]))
    if a:
        L.adj_toggle(a["id"])
        c = L.adj(a["id"])["cancelled"]
        await edit(q, f"{'↩️ Отменено' if c else '✅ Восстановлено'}: {adj_ti
```

```
                    K([[B("Вернуть" if c else "↩️ Отменить", f"ac:{a['id']}")]

# ── yangi odam so'rovi / adminlar
elif A == "uo" and uid == OWNER:
    tid = int(p[1])
    L.upsert_user(tid, app.bot_data.get("names", {}).get(tid, ""), "owner", L
    await edit(q, "✅ Это ваш второй аккаунт: видит всё, включая Cashliy и Ar
    await send(bot, tid, "✅ Принято. Нажмите /start")
elif A in ("ul", "ua", "ux", "un"):
    tid = int(p[1])
    if L.user(tid):
        await edit(q, "Уже обработано.")
        return
    name = app.bot_data.get("names", {}).get(tid, "")
    if A == "ul":
        wid = int(p[2])
        L.upsert_user(tid, name, "worker", wid)
        await edit(q, f"✅ Привязан к: {L.worker(wid)['name']}\n(зарплату пол
        await send(bot, tid, "✅ Принято.")
    elif A == "ua":
        L.upsert_user(tid, name, "admin", None)
        await edit(q, "✅ Назначен админом")
        await send(bot, tid, "✅ Принято. Нажмите /start")
    elif A == "un":
        ctx.user_data["st"] = ("newlink", tid)
        await edit(q, "Напишите новое имя обработчика — привяжу к нему этого
    else:
        await edit(q, "🚫 Отказано")
elif d == "adm:menu":
    await show_admins(q)
elif d == "adm:new":
    ctx.user_data["st"] = ("addadmin",)
    await edit(q, "Напишите Telegram ID нового админа (число):", K([menu_btn(
elif A == "adm" and p[1] == "u":
    u = L.user(int(p[2]))
    if u:
        wk = L.worker(u["worker_id"])["name"] if u["worker_id"] else "—"
        role = {"owner": "это вы (Cashliy)", "admin": "админ", "worker": "обы
        rows = [[B("👑 Снять админа" if u["role"] == "admin" else "👑 Сделать
                 [B("🔗 Привязать к обработчику", f"adm:l:{u['tg_id']}")], [B(
        if uid == OWNER:
            rows.insert(1, [B("👑 Снять «это я»" if u["role"] == "owner" else
        await edit(q, f"{u['name'] or u['tg_id']} (ID {u['tg_id']})\nРоль: {r
elif A == "adm" and p[1] == "o" and uid == OWNER:
    u = L.user(int(p[2]))
    if u:
```

```
            if u["role"] == "owner":
                L.set_role(u["tg_id"], "worker")
            else:
                L.upsert_user(u["tg_id"], u["name"], "owner", L.self_worker()["id
                await send(bot, u["tg_id"], "👑 Принято. Нажмите /start")
            await show_admins(q)
    elif A == "adm" and p[1] == "t":
        u = L.user(int(p[2]))
        if u and u["role"] != "owner":
            new = "worker" if u["role"] == "admin" else "admin"
            L.set_role(u["tg_id"], new)
            await send(bot, u["tg_id"], "👑 Вам выданы права админа. Нажмите /sta
        await show_admins(q)
    elif A == "adm" and p[1] == "l":
        rows = [[B(w["name"], f"adm:w:{p[2]}:{w['id']}")] for w in vis(uid, ("wor
        await edit(q, "К какому обработчику привязать?", K(rows))
    elif A == "adm" and p[1] == "w":
        L.set_user_worker(int(p[2]), int(p[3]))
        await show_admins(q)
    elif A == "adm" and p[1] == "d":
        u0 = L.user(int(p[2]))
        if u0 and (u0["role"] != "owner" or uid == OWNER):
            L.del_user(int(p[2]))
        await show_admins(q)


# ─────────────────────────── matn kiritish (holatlar) ──────────────────────────
def to_num(t):
    try:
        return float(t.strip().replace(",", ".").replace("$", "").replace("%", ""
    except ValueError:
        return None


def price_args(text):
    nums = [to_num(t) for t in text.split()]
    if None in nums or len(nums) not in (2, 3):
        return None
    return (int(nums[0]), int(nums[0]), nums[1]) if len(nums) == 2 else (int(nums


async def manual_report(app, text, uid):
    """Egasi/admin botga o'zi hisobot tashlaydi (eski hisobotlar). Tushunmasa tug
    tag, body = L.split_tag(text)
    checks = L.report_checks(body, L.today(), is_owner(uid))
    if not checks:
```

```
        return False
    named = L.worker_by_name(text)
    acct = tag or named or (L.self_worker()["id"] if is_owner(uid) else None)
    await submit_report(app, acct, None, 0, None, uid, "вручную", text, checks)
    return True


async def on_private_text(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    u, msg = update.effective_user, update.message
    if not is_staff(u.id):
        return                                          # qabul qilinmagan odam b
    text = (msg.text or "").strip()
    app = ctx.application

    # narx savoliga javob
    pq = app.bot_data.get("pq", {})
    key = None
    rc = getattr(msg, "reply_to_message", None)
    if rc and (msg.chat_id, rc.message_id) in pq:
        key = (msg.chat_id, rc.message_id)
    elif "st" not in ctx.user_data and to_num(text) is not None and len({v for v
        key = next((k for k in pq if k[0] == msg.chat_id), None)
    if key is not None:
        v = to_num(text)
        if v is None or v <= 0:
            await msg.reply_text("Нужно число, например 12. Ответьте ещё раз на т
            return
        rid, amount = pq[key]
        for k in [k for k, val in pq.items() if val == (rid, amount)]:
            pq.pop(k, None)
        L.add_price(amount, amount, v)
        await msg.reply_text(f"✅ Запомнил: {amount:,} = {L.money(v)}$")
        for r in L.pending_reports():
            need = L.report_needs(r)
            if need is None:
                await finalize(app, r)
            elif need[0] == "price" and not any(val == (r, need[1]) for val in pq
                await advance(app, r)
        return

    st = ctx.user_data.get("st")
    if not st:
        if L.has_kind(text) and not L.report_checks(text, L.today(), True):
            await handle_adjust(app, text, L.today(), 0, None)
        elif not await manual_report(app, text, u.id):
            await msg.reply_text(MAIN_TEXT, reply_markup=main_kb(u.id))
```

```
        return
    act = st[0]

    async def card_msg(wid):
        w = L.worker(wid)
        await msg.reply_text(card(w), reply_markup=card_kb(w, u.id))

    if act == "newname":
        if not text or L.name_taken(text):
            await msg.reply_text("Такое имя уже есть или пусто. Напишите другое:"
            return
        ctx.user_data["st"] = ("newpct", text)
        await msg.reply_text(f"Какой процент у {text}? Напишите число (например 3
    elif act == "newpct":
        v = to_num(text)
        if v is None or not 0 <= v <= 100:
            await msg.reply_text("Нужно число от 0 до 100:")
            return
        wid = L.add_worker(st[1], v)
        ctx.user_data.pop("st")
        await msg.reply_text(f"✅ Добавлен: {st[1]} · {L.money(v)}%")
        await card_msg(wid)
    elif act == "newlink":
        if not text or L.name_taken(text):
            await msg.reply_text("Такое имя уже есть или пусто. Напишите другое:"
            return
        ctx.user_data["st"] = ("newlinkpct", st[1], text)
        await msg.reply_text(f"Какой процент у {text}? Напишите число (например 3
    elif act == "newlinkpct":
        v = to_num(text)
        if v is None or not 0 <= v <= 100:
            await msg.reply_text("Нужно число от 0 до 100:")
            return
        ctx.user_data.pop("st")
        wid = L.add_worker(st[2], v)
        L.upsert_user(st[1], app.bot_data.get("names", {}).get(st[1], ""), "worke
        await msg.reply_text(f"✅ Создан {st[2]} ({L.money(v)}%) и привязан к чел
        await send(ctx.bot, st[1], "✅ Принято.")
    elif act == "pct":
        v = to_num(text)
        if v is None or not 0 <= v <= 100:
            await msg.reply_text("Нужно число от 0 до 100:")
            return
        ctx.user_data.pop("st")
        wid = st[1]
        await msg.reply_text(f"С какого дня новый процент {L.money(v)}%? (прошлые
```

```
                        reply_markup=K([[B("С понедельника этой недели", f"p
                                        [B("С сегодня", f"pcd:{wid}:{v}:td")
                                        [B("За всё время", f"pcd:{wid}:{v}:a
elif act in ("fine", "bonus", "bonusminus", "adv"):
    v = to_num(text)
    if v is None or v <= 0:
        await msg.reply_text("Нужно число больше нуля, например 5:")
        return
    ctx.user_data.pop("st")
    kind = {"fine": "fine", "bonus": "bonus", "bonusminus": "bonus", "adv": "
    aid = L.add_adj(st[1], kind, -v if act == "bonusminus" else v, L.iso(L.to
    await msg.reply_text(f"✅ Записано: {adj_title(L.adj(aid))}", reply_marku
elif act == "adjamt":
    v = to_num(text)
    if v is None or v <= 0:
        await msg.reply_text("Нужно число больше нуля, например 5:")
        return
    ctx.user_data.pop("st")
    L.adj_set(st[1], usd=v)
    t2, kb = adj_view(st[1], u.id)
    await msg.reply_text(t2, reply_markup=kb)
elif act == "trpct":
    v = to_num(text)
    if v is None or not 0 < v <= 100:
        await msg.reply_text("Нужно число от 1 до 100:")
        return
    ctx.user_data.pop("st")
    _, wid, day, to = st
    tid = L.add_transfer(day, wid, to, v)
    await msg.reply_text(f"✅ Перенос: {L.worker(wid)['name']} → {L.worker(tc
                         f"{L.money(L.r2(L.day_base(wid, day) * v / 100))}$",
                         reply_markup=K([[B("↩️ Отменить перенос", f"trt:{tid
elif act == "fxadd":
    v = to_num(text)
    if v is None or v < 1:
        await msg.reply_text("Нужна сумма чека, например 1950:")
        return
    ctx.user_data.pop("st")
    L.add_check(st[1], st[2], int(v))
    await msg.reply_text(f"✅ Чек {int(v)} добавлен.", reply_markup=K([[B("✏️
elif act == "chkamt":
    v = to_num(text)
    if v is None or v < 1:
        await msg.reply_text("Нужна сумма чека, например 1950:")
        return
    ctx.user_data.pop("st")
```

```
        c = L.check_row(st[1])
        if c and c["acct"] == st[2]:
            L.edit_check_amount(st[1], int(v))
        await msg.reply_text("✅ Сумма изменена.", reply_markup=K([[B("✏️ К чека
    elif act == "pdate":
        d0 = L.parse_date_text(text, L.today())
        if not d0:
            await msg.reply_text("Не понял дату. Пример: 05.10")
            return
        ctx.user_data.pop("st")
        L.x("update checks set rd=? where report_id=? and rd is null", (L.iso(d0)
        await msg.reply_text(f"✅ Дата: {L.dshort(L.iso(d0))}")
        await advance(app, st[1])
    elif act == "datepick":
        ds = L.parse_dates(text, L.today())
        if not ds:
            await msg.reply_text("Не понял дату. Пример: 05.10 или 05.10-11.10")
            return
        ctx.user_data.pop("st")
        key = L.iso(ds[0]) if len(ds) == 1 else f"{L.iso(min(ds[0], ds[1]))}_{L.i
        if st[1]:
            v = view_brief(u.id, st[1], key)
            if v:
                await msg.reply_text(v[0], reply_markup=v[1])
            else:
                await msg.reply_text("За этот период отчётов нет.", reply_markup=
        else:
            s, e = L.period_range(key)
            b = L.calc_bundle(s, e)
            if not any(L.has_data(r) and can_see(u.id, r["w"]["id"]) for r in b["
                await msg.reply_text(f"За {L.label(s, e)} отчётов нет.", reply_ma
                return
            rows_ = [[B(r["w"]["name"], f"rep:{key}:{r['w']['id']}")] for r in (b
                      if L.has_data(r) and can_see(u.id, r["w"]["id"]) and r["kind
            await msg.reply_text(L.overall_text(b, L.label(s, e), is_owner(u.id))
    elif act == "rename":
        ctx.user_data.pop("st")
        try:
            L.rename_worker(st[1], text)
        except Exception:
            await msg.reply_text("Такое имя уже есть.")
            return
        await card_msg(st[1])
    elif act == "replace":
        old = L.worker(st[1])
        if not text or L.name_taken(text):
```

```
            await msg.reply_text("Такое имя уже есть или пусто. Напишите другое:"
            return
        ctx.user_data.pop("st")
        pct = L.percent_on(old["id"], L.today())
        wid = L.add_worker(text, pct)
        L.set_active(old["id"], 0)
        await msg.reply_text(f"✅ {text} вместо {old['name']} · {L.money(pct)}% (
        await card_msg(wid)
    elif act in ("priceadd", "pricedit"):
        args = price_args(text)
        if not args:
            await msg.reply_text("Формат: «сумма цена» (5200 20) или «от до цена»
            return
        ctx.user_data.pop("st")
        if act == "priceadd":
            L.add_price(*args)
        else:
            L.update_price(st[1], *args)
        await msg.reply_text("✅ Готово.", reply_markup=K([[B("💲 Прайс", "pr:mer
        for r in L.pending_reports():
            if L.report_needs(r) is None:
                await finalize(app, r)
    elif act == "shpct":
        v = to_num(text)
        if v is None or not 0 < v <= 100:
            await msg.reply_text("Нужно число от 1 до 100:")
            return
        ctx.user_data.pop("st")
        L.update_share(st[1], pct=v)
        await msg.reply_text("✅ Готово.", reply_markup=K([[B("🤝 К доле", f"shc:
    elif act == "shnew":
        v = to_num(text)
        if v is None or not 0 < v <= 100:
            await msg.reply_text("Нужно число от 1 до 100:")
            return
        ctx.user_data.pop("st")
        sid = L.add_share(st[1], st[2], v, "zp", "deduct", L.iso(L.week_start()))
        await msg.reply_text("✅ Доля добавлена.", reply_markup=K([[B("🤝 К доле'
    elif act == "addadmin":
        v = to_num(text)
        if v is None:
            await msg.reply_text("Нужен числовой ID:")
            return
        ctx.user_data.pop("st")
        tid = int(v)
        if not is_owner(tid):
```

```
            u0 = L.user(tid)
            L.upsert_user(tid, (u0["name"] if u0 else ""), "admin", (u0["worker_i
            await send(ctx.bot, tid, "👑 Вам выданы права админа. Нажмите /start"
        await msg.reply_text("✅ Админ добавлен.", reply_markup=K([[B("🛡️ Админы'


async def on_error(update, ctx: ContextTypes.DEFAULT_TYPE):
    log.exception("error: %s", ctx.error)
    try:
        await ctx.bot.send_message(OWNER, f"⚠️ Ошибка бота: {str(ctx.error)[:300]
    except Exception:
        pass


async def post_init(app: Application):
    for rid in L.pending_reports():
        await advance(app, rid)
    for aid in L.pending_adjs():
        await ask_adj(app, aid)
    await ask_unassigned(app)


def main():
    L.db()
    app = Application.builder().token(TOKEN).post_init(post_init).build()
    app.add_handler(CommandHandler("start", cmd_start))
    app.add_handler(CommandHandler("id", cmd_id))
    app.add_handler(CallbackQueryHandler(on_cb))
    app.add_handler(ChatMemberHandler(on_my_chat_member, ChatMemberHandler.MY_CHA
    app.add_handler(MessageHandler(filters.ChatType.PRIVATE & filters.TEXT & ~fil
    app.add_handler(MessageHandler((filters.ChatType.GROUPS | filters.ChatType.CH
    app.add_error_handler(on_error)
    app.run_polling(allowed_updates=Update.ALL_TYPES)


if __name__ == "__main__":
    main()
```